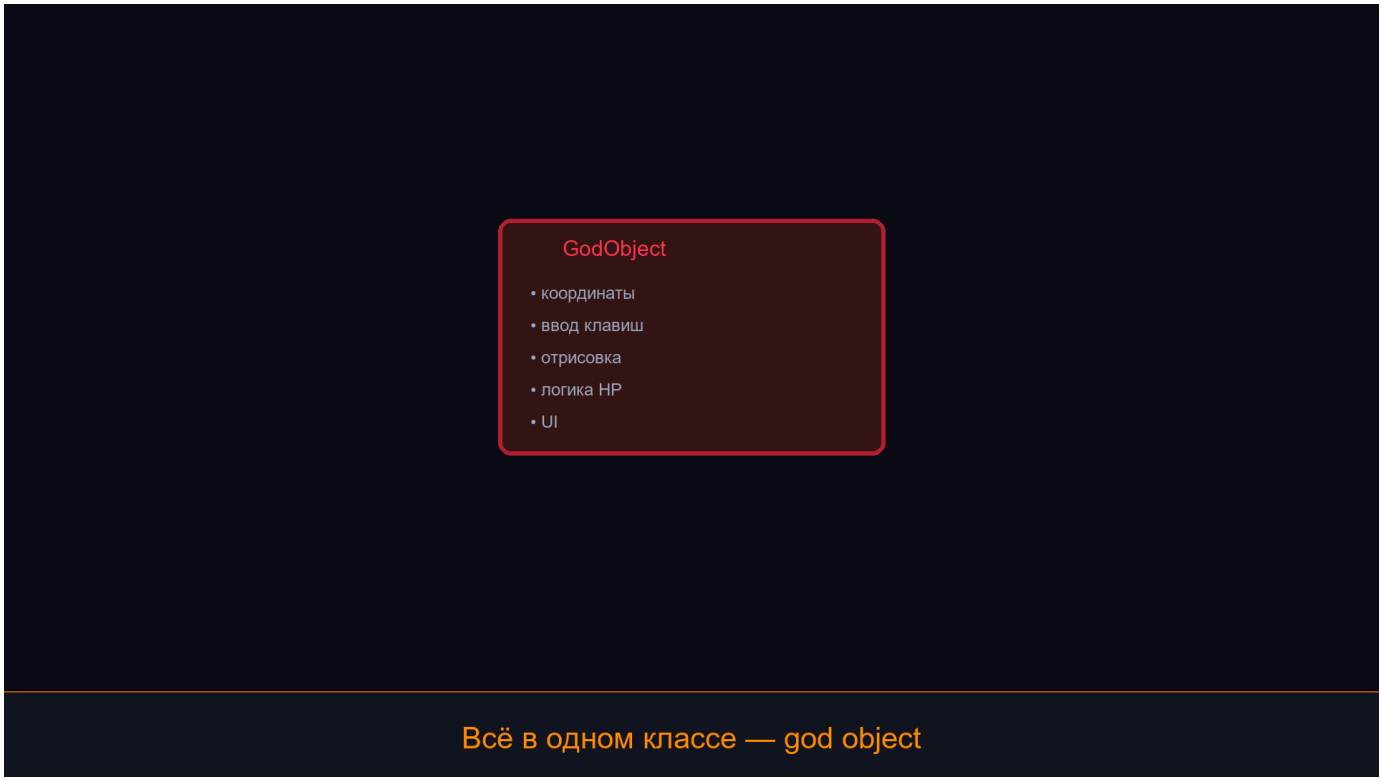
MVC, MVP и MVVM

MVC, MVP и MVVM



Сюда входят MVC, MVP, MVVM - разные способы разделить игру на три части: данные, отображение, управление.

Проблема: всё в одном месте

Допустим, мы пишем игру. Начинаем просто - один класс, в нём координаты игрока, здоровье, обработка клавиш, отрисовка.

Python C#

```python
class Game:
    def __init__(self):
        self.player_x = 100
        self.player_y = 200
        self.player_hp = 100
        self.player_sprite = pygame.image.load("player.png")

    def update(self):
```

```
        keys = pygame.key.get_pressed()
        if keys[pygame.K_LEFT]:
            self.player_x -= 5
        if keys[pygame.K_RIGHT]:
            self.player_x += 5
        # ... 500 more lines for enemies, items, UI

    def draw(self, screen):
        screen.blit(self.player_sprite, (self.player_x, self.player_y))
        pygame.draw.rect(screen, RED, (10, 10, self.player_hp, 20))
        # ... 300 more lines
```

Работает? Работает. Проблема появляется, когда проект растёт. Добавили врагов - `update()` раздулся. Добавили меню - `draw()` раздулся. Хотим поменять управление - копаемся в том же файле, где игровая логика и отрисовка. Это god object - классический антипаттерн.

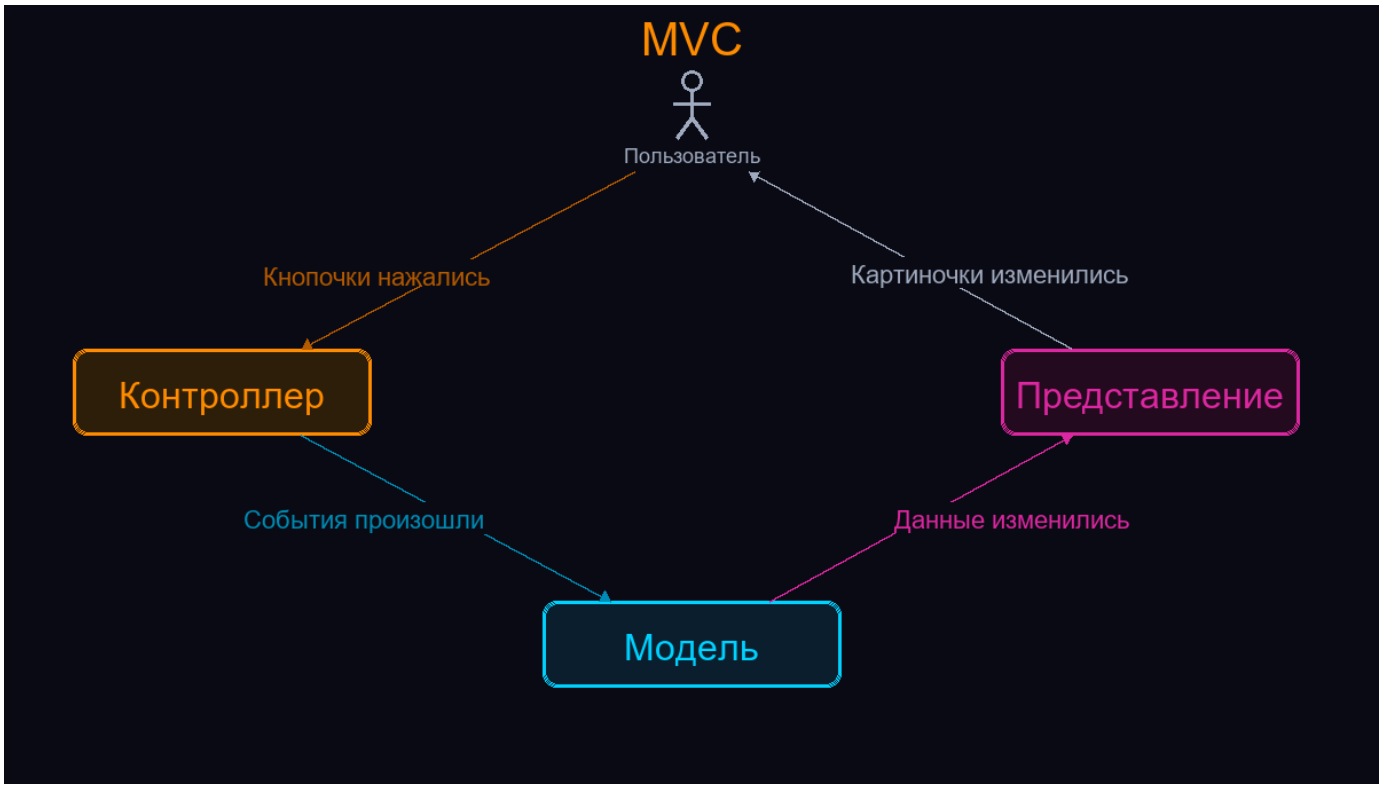
А что, если разделить ответственность?

MVC - Model-View-Controller

Model (модель) - хранит данные и бизнес-логику. Координаты, здоровье, столкновения. Ничего не знает про отрисовку и ввод.

View (представление) - отрисовка. Берёт данные из модели и рисует. Не содержит игровой логики.

Controller (контроллер) - обработка ввода. Принимает нажатия клавиш и говорит модели, что делать.



Т.е. вместо одного класса, который делает всё, мы получаем три, каждый со своей зоной ответственности.

Python C#

```python
class PlayerModel:
    def __init__(self, x, y, hp):
        self.x = x
        self.y = y
        self.hp = hp
        self.speed = 5

    def move(self, dx, dy):
        self.x += dx * self.speed
        self.y += dy * self.speed

    def take_damage(self, amount):
        self.hp = max(0, self.hp - amount)


class PlayerView:
    def __init__(self, sprite):
        self.sprite = sprite

    def draw(self, screen, model):
        screen.blit(self.sprite, (model.x, model.y))
        bar_width = int(model.hp / 100 * 200)
        pygame.draw.rect(screen, RED, (10, 10, bar_width, 20))


class PlayerController:
    def __init__(self, model):
        self.model = model

    def handle_input(self):
        keys = pygame.key.get_pressed()
        dx = keys[pygame.K_RIGHT] - keys[pygame.K_LEFT]  # bool: 1 or 0, d:
        dy = keys[pygame.K_DOWN] - keys[pygame.K_UP]
        self.model.move(dx, dy)


# game loop
model = PlayerModel(100, 200, 100)
view = PlayerView(player_sprite)
controller = PlayerController(model)

while running:
    for event in pygame.event.get():
```

```
        if event.type == pygame.QUIT:
            running = False
    controller.handle_input()
    view.draw(screen, model)
    pygame.display.flip()
```

Теперь каждая часть занимается своим делом:

- Что хранит координаты игрока? `PlayerModel` .
- Где логика перемещения? `PlayerModel` .
- Где определяются текстуры и отрисовка? `PlayerView` .
- Где обработка клавиатуры? `PlayerController` .

Хотим поменять управление на геймпад? Меняем только контроллер. Хотим другой спрайт? Меняем только представление. Модель не трогаем.

Модель не знает, что существует View или Controller. Она просто хранит данные и предоставляет методы. Это ключевой принцип - **модель независима**.

Как модель сообщает об изменениях?

В примере выше View получает модель напрямую и сам читает данные - это самый простой вариант (polling). Альтернатива - события: модель публикует событие ("здоровье изменилось"), View подписывается. Это паттерн Observer. Удобно, когда данные меняются редко или нужно уведомить несколько подписчиков.

Что нам даёт такое разбиение?

Мы чётко разграничиваем зоны ответственности. Изменения в одном месте не ломают другое. Каждую часть можно тестировать отдельно - модель без отрисовки, контроллер без модели.

MVC не говорит, как именно реализован `PlayerModel` внутри. Класс? Структура? ООП? EC/ECS? Это уже отдельный вопрос.

MVP - Model-View-Presenter

MVP - развитие идеи MVC. Основное отличие: View и Model **не общаются напрямую**. Вся связь идёт через Presenter.

- **Model** - так же отвечает за данные и логику. Не знает о представлении.
- **View** - отображает данные. Но теперь не обращается к модели сам, а делегирует всё Presenter.
- **Presenter** - посредник. Получает события от View, обновляет Model, берёт данные из Model и передаёт обратно в View.

Грубо говоря, в MVC View мог сам читать данные модели. В MVP - нет. Presenter полностью управляет потоком данных. Это упрощает тестирование: подменяем View на заглушку и тестируем только Presenter + Model.

Ключевое отличие от MVC - `PlayerView.show()` принимает уже готовые данные, а не объект модели:

Python C#

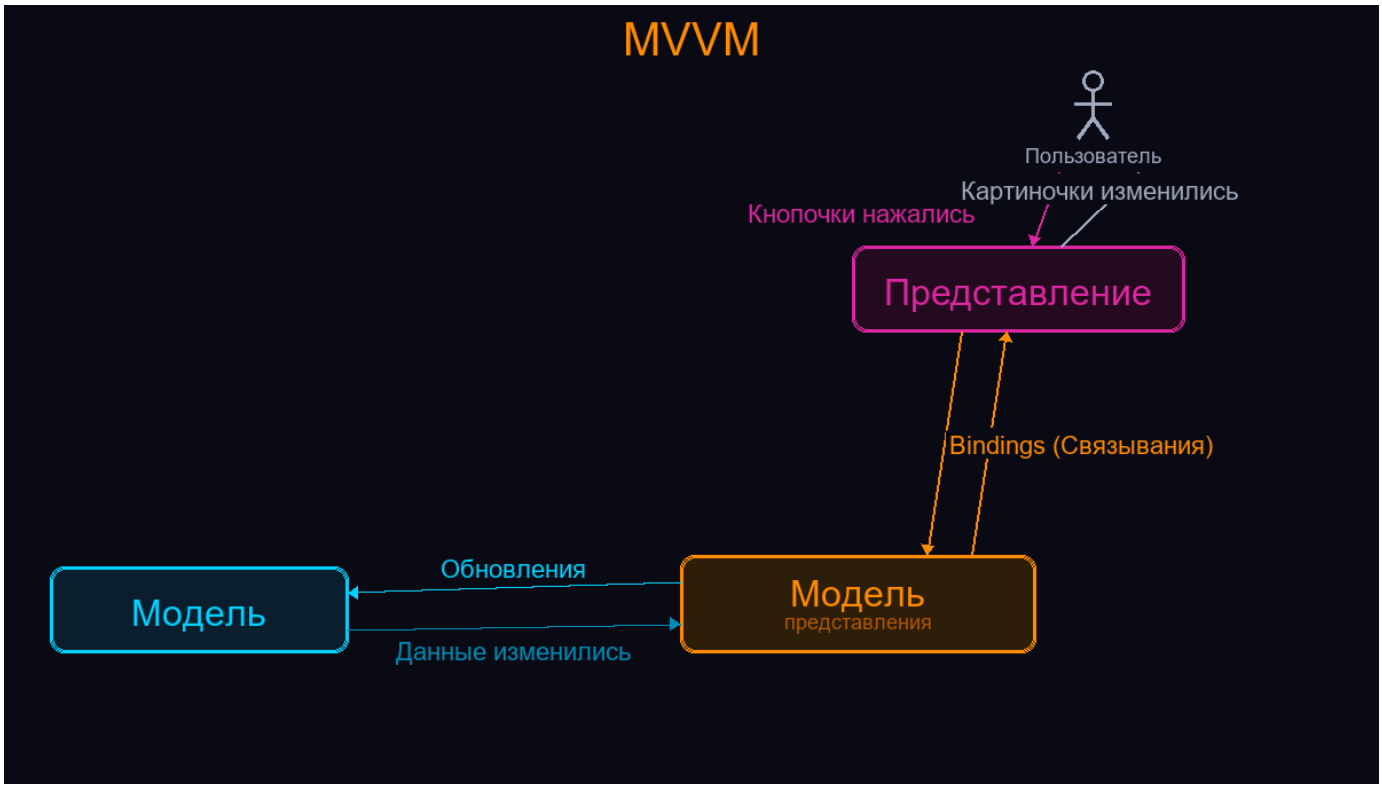
```python
class PlayerView:
    def __init__(self, sprite):
        self.sprite = sprite

    def show(self, screen, x, y, hp):
        screen.blit(self.sprite, (x, y))
        bar_width = int(hp / 100 * 200)
        pygame.draw.rect(screen, RED, (10, 10, bar_width, 20))


class PlayerPresenter:
    def __init__(self, model, view):
        self.model = model

```

```
        self.view = view

    def update(self):
        keys = pygame.key.get_pressed()
        dx = keys[pygame.K_RIGHT] - keys[pygame.K_LEFT]
        dy = keys[pygame.K_DOWN] - keys[pygame.K_UP]
        self.model.move(dx, dy)

    def render(self, screen):
        # Presenter reads from Model and passes to View
        self.view.show(screen, self.model.x, self.model.y, self.model.hp)


# game loop
model = PlayerModel(100, 200, 100)
view = PlayerView(player_sprite)
presenter = PlayerPresenter(model, view)

while running:
    for event in pygame.event.get():
        if event.type == pygame.QUIT:
            running = False
    presenter.update()
    presenter.render(screen)
    pygame.display.flip()
```

View больше не знает о существовании модели. Это и есть главное отличие от MVC.

MVVM - Model-View-ViewModel

MVVM добавляет концепцию **связывания данных (data binding)**: UI автоматически обновляется при изменении ViewModel, и наоборот.



- **Model** - данные и логика.
- **View** - UI, отображение.
- **ViewModel** - посредник с автоматической синхронизацией. View привязывается к свойствам ViewModel, и при изменении данных UI обновляется сам.

В MVC и MVP такую связь мы создаём вручную - события, колбэки. В MVVM это автоматизировано фреймворком. Чаще встречается в WPF, UWP, Unity UI Toolkit.

Для проекта на pygame или MonoGame MVVM скорее всего будет избыточным. MVC или MVP - более практичный выбор.

А что, если представления нет?

Ок. Грубо говоря, каждый объект мы разбиваем на три части. Но зачем представление чему-то, что не отображается?

Допустим, система сохранений:

- **Загрузка данных**: читает файл с диска
- **Выгрузка данных**: конвертирует состояние игры в JSON и сохраняет
- **Автоматизация**: чекпоинты по миру, без меню сохранения

Где тут View? Нигде - пользователь не видит процесс сохранения. Model хранит логику сериализации. Controller определяет, когда сохраняться. А представления нет.

MVC применяется к тому, что отображается на экране. Для остального - достаточно модели и контроллера.

Сравнение

	MVC	MVP	MVVM
Связь View<->Model	View читает Model напрямую	Только через Presenter	Через data binding
Кто управляет View?	Controller + View сам	Presenter полностью	ViewModel через привязки
Тестируемость	Средняя	Высокая	Высокая
Сложность	Низкая	Средняя	Выше (нужен фреймворк)
Где применяется	Игры, простые приложения	Мобильные приложения	WPF, Unity UI Toolkit

Не существует "лучшего" паттерна. Для небольшой игры MVC - самый прямой путь. MVP - если хотите чистого разделения. MVVM - если фреймворк поддерживает data binding.

А как именно реализовать Model внутри - через ООП, EC или ECS, конечные автоматы для управления состояниями сцен - это уже следующий вопрос. MVC говорит **что** разделять, а не **как** реализовывать каждую часть.

Подробнее об архитектуре в целом - на отдельной странице.

Что дальше?

Тема	Зачем
Архитектура	Общая картина: как связать все части проекта
Событийная модель	Observer и события - способ связать Model и View без жёстких зависимостей
ООП	Наследование, композиция и когда что выбирать
EC и ECS	Альтернативный подход к организации данных внутри Model
Конечные автоматы	Управление состояниями сцен и объектов

Powered by Wiki.js